HW2

- **Submission date: 02/10/2025 (11:59 PM)**
- Revise the topics discussed in Class *thoroughly* before attempting the assignment.

1. **NumPy.** (Review the NumPy tutorial thoroughly before answering. Additionally, refer to Sec. 1.3 in the Scientific Python lectures (link)).
   a. Create a NumPy array `np_array1` of integers from 0 to 25 and reshape it into a (5,5) matrix.
   b. Create a NumPy array `np_array2` of size 25 of equally spaced numbers from 5.0 to 10.0 (both end points inclusive) and print the array.
   This clearly shows too many digits after the decimal point. Write a function called `matrix_round()` that takes in a NumPy array and a desired precision, and returns another array of the same shape with all entries rounded to the desired precision.
   E.g. If the input is a (4, 2) matrix of the form

```
array([[ 7.        ,  7.42857143],
       [ 7.85714286,  8.28571429],
       [ 8.71428571,  9.14285714],
       [ 9.57142857, 10.        ]])
```

   The output of passing this matrix and a precision of 3 must return (you don't have to print the precision)

```
Precision = 3
array([[ 7.   ,  7.429],
       [ 7.857,  8.286],
       [ 8.714,  9.143],
       [ 9.571, 10.   ]])
```

   Remember, the only two inputs to the function should be the NumPy array and the desired precision.
   c. Read the properties of matrix inversion in the class notes. Compute the inverse of the matrix V with the following entries (the `linalg` package might help here)

```
array([[  1,   1,   1,   1,   1],
       [  1,   2,   4,   8,  16],
       [  1,   3,   9,  27,  81],
       [  1,   4,  16,  64, 256],
       [  1,   5,  25, 125, 625]])
```

   Print V^{-1} to 3 decimal places. Compute $V^{-1}V$ and $V\,V^{-1}$ and print them to 3 decimal places. What is the name given to the matrices $V^{-1}V$ and $V\,V^{-1}$? What NumPy command is used to generate such matrices?

2. **Solving systems of equations in NumPy.**
   a. You are given the following system of equations

$$3x - y + 2z = 3$$
$$-y + z = 5$$
$$2x - 3z = -4$$

Express the system as $A\boldsymbol{b} = \boldsymbol{c}$, where A is a (3,3) coefficient matrix, $\boldsymbol{b} = [x, y, z]^T$ is a (3,1) vector of all the unknowns and $\boldsymbol{c}$ is another (3,1) vector containing the right-hand side of the system of equations**.**

b. Print A and A^{-1} to 3 decimal places. How would you use A^{-1} to solve for the vector $\boldsymbol{b}$?
c. What is the value of $\boldsymbol{b}$? Print to 3 decimal places.
d. **Overdetermined systems.** When the number of equations is larger than the number of unknowns, the system of equations is said to be "overdetermined." Consider the following system of equations.

$$\begin{aligned} x + 4y + 7z &= 3 \\ 2x - 5y + 8z &= 5 \\ 3x + 6y - 9z &= -4 \\ -x + 3y &= 1 \\ y &= -2 \end{aligned}$$

Once again, write this system of equations in the form $A\boldsymbol{b} = \boldsymbol{c}$. Is the matrix A invertible? What happens when you try to invert A?

e. In such situations, a special inverse called the "Pseudoinverse" is used. The pseudoinverse of A is denoted by A^+. Using the `pinv()` function in the `linalg` package, print A^+A up to 3 decimal places.
f. Now, how would you use A^+ to solve $A\boldsymbol{b} = \boldsymbol{c}$? Is this solution exact or approximate?

3. **MatplotLib.** (Review the MatplotLib tutorial thoroughly before answering. Additionally, refer to Sec. 1.4 in the Scientific Python lectures (link)) solve the problems related to the MatplotLib library in the Jupyter Notebooks named `CS767_HW2_matplotlib_questions_Part1.ipynb` and `CS767_HW2_matplotlib_questions_Part2.ipynb`.

4. **Cross Validation.** (References: [GBC16] Sec. 5.2 and 5.3, [BB22] Sec. 1.2.4 – 1.2.6)
   a. Using the function $\mathcal{L}(\boldsymbol{w}, \lambda)$ discussed in class, explain why the optimal regularization coefficient, λ^* cannot be learned from $\mathcal{D}_{train}$, and requires a separate validation set. Assume we're using 2-norm regularization. Recall that we defined

$$\mathcal{L}(\boldsymbol{w}, \lambda) = \frac{1}{\mathcal{N}_{train}} \sum_{i=1}^{\mathcal{N}_{train}} (y_i - \hat{y}_i)^2 + \lambda \|\boldsymbol{w}\|_2^2$$

   b. Consider the steps involved in cross-validation for the specific case of polynomial regression.
      - Assume the dataset contains 2-dimensional features (i.e., $k = 2$), and we're using polynomials of degree $d = 3$. How many terms will the predictor $h(\boldsymbol{w}, \boldsymbol{x})$ have?
      - Write the full form of the predictor $\hat{y} = h(\boldsymbol{w}, \boldsymbol{x})$ showing all the constant, linear, quadratic, and cubic terms (including interaction terms).
      - What happens to the λ^* that is found during the cross-validation process? With what data and loss function are the final weights computed before the testing phase begins?